Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-208БВ-25

Студент: Никонова А.Э.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 04.11.26

Москва, 2026

Постановка задачи

Вариант 16.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или через отображаемые файлы (memory-mapped files). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Общий метод и алгоритм решения

Использованные системные вызовы:

- pid_t fork(void); - создаёт дочерний процесс (копию текущего). В родителе возвращает PID ребёнка, в ребёнке - 0, при ошибке - -1. Используется в server.c для запуска client.
- int execl(const char *path, const char *arg, ..., NULL); - заменяет образ текущего процесса новой программой. В server.c (в ребёнке) запускает ./client, передавая имена shared memory и семафоров аргументами. При успехе не возвращается.
- pid_t getpid(void); - возвращает PID текущего процесса. В server.c используется для формирования уникальных имён /lab3_shm1_<pid>, /lab3_s1r_<pid> и т.д.
- pid_t wait(int *status); - ожидает завершения любого дочернего процесса, забирает зомби. В server.c после цикла чтения ошибок.
- ssize_t read(int fd, void *buf, size_t count); - читает до count байт из fd. Используется для посимвольного чтения stdin в read_line и в цикле ввода имени файла (server.c).
- ssize_t write(int fd, const void *buf, size_t count); - пишет до count байт в fd. Используется везде: приглашение, сообщения об ошибках в stderr, вывод валидных строк в stdout (файл) в client.c, вывод ошибок из SHM2 в server.c.
- int open(const char *pathname, int flags, mode_t mode); - открывает/создаёт файл, возвращает fd. В server.c открывает выходной файл по имени, введённому пользователем.
- int close(int fd); - закрывает fd. Используется после open, shm_open, dup2.
- int dup2(int oldfd, int newfd); - дублирует oldfd в newfd, закрывая старый newfd. В server.c: dup2(fd, STDOUT_FILENO)- перенаправляет stdout в файл; ребёнок наследует это перенаправление.
- int shm_open(const char *name, int oflag, mode_t mode); - создаёт/открывает **именованный объект разделяемой памяти**. В server.c - с O_CREAT|O_RDWR (создаёт), в client.c - только O_RDWR (открывает существующий). Возвращает fd.
- int ftruncate(int fd, off_t length); - задаёт размер объекту. Новый объект shm_open имеет размер 0, без ftruncate mmap/доступ дадут SIGBUS. В server.c - sizeof(struct slot).
- void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset); — отображает объект в адресное пространство. PROT_READ|PROT_WRITE, MAP_SHARED, fd от shm_open. В обоих файлах.

- int munmap(void *addr, size_t length); - снимает отображение. В конце обоих процессов.
- int shm_unlink(const char *name); - удаляет имя объекта shared memory из namespace. В server.c после munmap (владелец - server).
- sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value); — создаёт/открывает именованный семафор. В server.c - с O_CREAT и начальным значением (0 для ready, 1 для free). В client.c - без O_CREAT (подключение к существующим).
- int sem_wait(sem_t *sem); - уменьшает счётчик; если 0 - блокируется. Обёрнут в must_sem_wait с обработкой EINTR.
- int sem_post(sem_t *sem); - увеличивает счётчик, будит ожидающих. Обёрнут в must_sem_post.
- int sem_close(sem_t *sem);- закрывает ссылку на семафор (аналог close для fd). В конце обоих процессов.
- int sem_unlink(const char *name); - удаляет имя семафора из namespace. В server.c после sem_close.

Программа реализована в виде двух отдельных исполняемых файлов: server (родительский процесс) и client (дочерний процесс). Вариант задания - 16: строка считается валидной, если она оканчивается на . или ;При запуске server запрашивает у пользователя имя файла, открывает его на запись и с помощью dup2 перенаправляет свой стандартный поток вывода в этот файл. После fork дочерний процесс наследует перенаправленный stdout, поэтому и server, и client в дальнейшем пишут свои результаты в один и тот же файл. Затем server формирует уникальные имена для двух объектов разделяемой памяти и четырёх семафоров, добавляя к префиксу PID текущего процесса (например, /lab3_shm1_12345). Это гарантирует, что два одновременных запуска программы не пересекутся. Через shm_open создаются два сегмента разделяемой памяти, размер каждого задаётся ftruncate, после чего оба сегмента отображаются в адресное пространство через mmap с флагом MAP_SHARED на файловые дескрипторы, возвращённые shm_open .Каждый сегмент имеет структуру struct slot, содержащую длину данных, флаг конца потока и буфер для самих данных. Первый сегмент (SHM1) используется для передачи строк от server к client, второй (SHM2) — для передачи сообщений об ошибках от client к server.Для каждого направления создаётся пара семафоров: ready (начальное значение 0) сигнализирует о наличии данных, free (начальное значение 1) - о том, что слот свободен и в него можно писать. Такая схема реализует классический шаблон producer/consumer с одним слотом. После подготовки ресурсов server вызывает fork, и в дочернем процессе через execl запускается программа client, которой имена shared memory и семафоров передаются аргументами командной строки. client открывает те же объекты через shm_open и sem_open (без O_CREAT), отображает их через mmap и входит в цикл обработки. server читает строки из стандартного ввода (терминала) и для каждой строки выполняет sem_wait(free), копирует строку в SHM1, выставляет длину и сбрасывает флаг EOF, после чего делает sem_post(ready). client, дождавшись sem_wait(ready), копирует строку из SHM1 в локальный буфер, немедленно освобождает слот через sem_post(free) и проверяет последний значащий символ строки. Если строка валидна (оканчивается на . или ;), client пишет её в свой stdout, который благодаря наследованию указывает в выходной файл. Если строка невалидна, client формирует сообщение вида INVALID: <строка>, дожидается sem_wait(free) на втором наборе семафоров, записывает сообщение в SHM2 и сигнализирует sem_post(ready).

Когда пользователь завершает ввод (Ctrl+D), server записывает в SHM1 маркер конца (eof = 1) и переходит к циклу чтения ошибок из SHM2. client, получив маркер EOF, отправляет аналогичный маркер в SHM2 и завершается. server читает все накопленные сообщения об ошибках из SHM2 и выводит их в свой stdout, то есть в тот же файл. В результате в выходном файле оказываются и валидные строки (их пишет client), и сообщения об ошибках (их пишет server), что соответствует схеме перенаправления из задания. По завершении работы server вызывает wait для ожидания дочернего процесса, снимает отображения через munmap, закрывает семафоры через sem_close и удаляет имена объектов через sem_unlink и shm_unlink. client выполняет только munmap и sem_close, поскольку владельцем ресурсов является server.

Все ключевые системные вызовы проверяются: open, dup2, shm_open, ftruncate, mmap, sem_open, fork, wait. При ошибке выводится сообщение в стандартный поток ошибок и программа завершается с ненулевым кодом. Для sem_wait и sem_post написаны обёртки must_sem_wait и must_sem_post: первая повторяет вызов при прерывании сигналом (EINTR), вторая проверяет результат. Запись в файл также проверяется на соответствие ожидаемому количеству байт.

Код программы

Server.c

```c
#define _GNU_SOURCE
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <semaphore.h>
#include <errno.h>
#define MAX_LINE 8192

struct slot {
    size_t len;
    int eof;
    char data[MAX_LINE];
};

static char *read_line(int fd)
{
    size_t cap = 128, len = 0;
    char *buf = malloc(cap);
    if (!buf) return NULL;
    char c; ssize_t n;
    while ((n = read(fd, &c, 1)) == 1) {
        if (len + 2 > cap) {
            cap *= 2;
            char *tmp = realloc(buf, cap);
            if (!tmp) { free(buf); return NULL; }
            buf = tmp;
        }
        buf[len++] = c;
```

```
        if (c == '\n') break;
    }
    if (n <= 0 && len == 0) { free(buf); return NULL; }
    buf[len] = '\0';
    return buf;
}

static void itoa_buf(int v, char *out)
{
    char tmp[16]; int t = 0, i = 0;
    if (v == 0) tmp[t++] = '0';
    while (v > 0) { tmp[t++] = '0' + v % 10; v /= 10; }
    while (t > 0) out[i++] = tmp[--t];
    out[i] = '\0';
}
static void must_sem_wait(sem_t *s)
{
    while (sem_wait(s) < 0) {
        if (errno == EINTR) continue;
        write(STDERR_FILENO, "sem_wait failed\n", 16);
        _exit(1);
    }
}

static void must_sem_post(sem_t *s)
{
    if (sem_post(s) < 0) {
        write(STDERR_FILENO, "sem_post failed\n", 16);
        _exit(1);
    }
}
int main(void)
{
    char name[512];
    write(STDOUT_FILENO, "Enter file name: ", 17);
    size_t i = 0; char c;
    while (read(STDIN_FILENO, &c, 1) == 1 && c != '\n' && i < sizeof(name) - 1)
        name[i++] = c;
    name[i] = '\0';

    int fd = open(name, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) { write(STDERR_FILENO, "open failed\n", 12); return 1; }
    if (dup2(fd, STDOUT_FILENO) < 0) { write(STDERR_FILENO, "dup2 failed\n", 12); return
1; }
    close(fd);

    int pid = (int)getpid();
    char num[16]; itoa_buf(pid, num);

    char shm1_name[64], shm2_name[64];
    char s1r_name[64], s1f_name[64], s2r_name[64], s2f_name[64];

    strcpy(shm1_name, "/lab3_shm1_"); strcat(shm1_name, num);
    strcpy(shm2_name, "/lab3_shm2_"); strcat(shm2_name, num);
```

```
    strcpy(s1r_name,  "/lab3_s1r_");  strcat(s1r_name,  num);
    strcpy(s1f_name,  "/lab3_s1f_");  strcat(s1f_name,  num);
    strcpy(s2r_name,  "/lab3_s2r_");  strcat(s2r_name,  num);
    strcpy(s2f_name,  "/lab3_s2f_");  strcat(s2f_name,  num);

    int shm1_fd = shm_open(shm1_name, O_CREAT | O_RDWR, 0600);
    int shm2_fd = shm_open(shm2_name, O_CREAT | O_RDWR, 0600);
    if (shm1_fd < 0 || shm2_fd < 0) {
        write(STDERR_FILENO, "shm_open failed\n", 16);
        return 1;
    }
    if (ftruncate(shm1_fd, sizeof(struct slot)) < 0 ||
        ftruncate(shm2_fd, sizeof(struct slot)) < 0) {
        write(STDERR_FILENO, "ftruncate failed\n", 17);
        return 1;
    }

    struct slot *to_client = mmap(NULL, sizeof(struct slot), PROT_READ | PROT_WRITE,
                                  MAP_SHARED, shm1_fd, 0);
    struct slot *to_server = mmap(NULL, sizeof(struct slot), PROT_READ | PROT_WRITE,
                                  MAP_SHARED, shm2_fd, 0);
    if (to_client == MAP_FAILED || to_server == MAP_FAILED) {
        write(STDERR_FILENO, "mmap failed\n", 12);
        return 1;
    }
    close(shm1_fd);
    close(shm2_fd);

    to_client->len = 0; to_client->eof = 0;
    to_server->len = 0; to_server->eof = 0;

    sem_t *s1_ready = sem_open(s1r_name, O_CREAT, 0600, 0);
    sem_t *s1_free  = sem_open(s1f_name, O_CREAT, 0600, 1);
    sem_t *s2_ready = sem_open(s2r_name, O_CREAT, 0600, 0);
    sem_t *s2_free  = sem_open(s2f_name, O_CREAT, 0600, 1);
    if (s1_ready == SEM_FAILED || s1_free == SEM_FAILED ||
        s2_ready == SEM_FAILED || s2_free == SEM_FAILED) {
        write(STDERR_FILENO, "sem_open failed\n", 16);
        return 1;
    }
    pid_t pidc = fork();
    if (pidc < 0) { write(STDERR_FILENO, "fork failed\n", 12); return 1; }

    if (pidc == 0) {
        execl("./client", "client",
              shm1_name, shm2_name, s1r_name, s1f_name, s2r_name, s2f_name, NULL);
        _exit(1);
    }

    char *line;
    while ((line = read_line(STDIN_FILENO)) != NULL) {
        size_t len = strlen(line);
        if (len > MAX_LINE) len = MAX_LINE;
```

```
        must_sem_wait(s1_free);
        memcpy(to_client->data, line, len);
        to_client->len = len;
        to_client->eof = 0;
        must_sem_post(s1_ready);

        free(line);
    }
    must_sem_wait(s1_free);
    to_client->len = 0;
    to_client->eof = 1;
    must_sem_post(s1_ready);
    for (;;) {
        must_sem_wait(s2_ready);
        size_t len = to_server->len;
        int eof = to_server->eof;
        if (len > 0 && write(STDOUT_FILENO, to_server->data, len) != (ssize_t)len) {
            write(STDERR_FILENO, "write failed\n", 13);
            _exit(1);
        }
        must_sem_post(s2_free);
        if (eof) break;
    }

    if (wait(NULL) < 0) {
        write(STDERR_FILENO, "wait failed\n", 12);
    }
    munmap(to_client, sizeof(struct slot));
    munmap(to_server, sizeof(struct slot));
    sem_close(s1_ready); sem_close(s1_free);
    sem_close(s2_ready); sem_close(s2_free);
    sem_unlink(s1r_name); sem_unlink(s1f_name);
    sem_unlink(s2r_name); sem_unlink(s2f_name);
    shm_unlink(shm1_name); shm_unlink(shm2_name);

    return 0;
}
```

Client.c

```
#define _GNU_SOURCE
#include <unistd.h>
#include <string.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <semaphore.h>
#include <errno.h>
#define MAX_LINE 8192

struct slot {
    size_t len;
    int eof;
```

```
    char data[MAX_LINE];
};
static void must_sem_wait(sem_t *s)
{
    while (sem_wait(s) < 0) {
        if (errno == EINTR) continue;
        write(STDERR_FILENO, "sem_wait failed\n", 16);
        _exit(1);
    }
}

static void must_sem_post(sem_t *s)
{
    if (sem_post(s) < 0) {
        write(STDERR_FILENO, "sem_post failed\n", 16);
        _exit(1);
    }
}
int main(int argc, char **argv)
{
    if (argc < 7) {
        write(STDERR_FILENO, "bad args\n", 9);
        return 1;
    }

    const char *shm1_name = argv[1];
    const char *shm2_name = argv[2];
    const char *s1r_name  = argv[3];
    const char *s1f_name  = argv[4];
    const char *s2r_name  = argv[5];
    const char *s2f_name  = argv[6];
    int shm1_fd = shm_open(shm1_name, O_RDWR, 0600);
    int shm2_fd = shm_open(shm2_name, O_RDWR, 0600);
    if (shm1_fd < 0 || shm2_fd < 0) return 1;

    struct slot *to_client = mmap(NULL, sizeof(struct slot), PROT_READ | PROT_WRITE,
                                  MAP_SHARED, shm1_fd, 0);
    struct slot *to_server = mmap(NULL, sizeof(struct slot), PROT_READ | PROT_WRITE,
                                  MAP_SHARED, shm2_fd, 0);
    if (to_client == MAP_FAILED || to_server == MAP_FAILED) return 1;
    close(shm1_fd);
    close(shm2_fd);

    sem_t *s1_ready = sem_open(s1r_name, 0);
    sem_t *s1_free  = sem_open(s1f_name, 0);
    sem_t *s2_ready = sem_open(s2r_name, 0);
    sem_t *s2_free  = sem_open(s2f_name, 0);
    if (s1_ready == SEM_FAILED || s1_free == SEM_FAILED ||
        s2_ready == SEM_FAILED || s2_free == SEM_FAILED) return 1;

    for (;;) {
        must_sem_wait(s1_ready);
        size_t len = to_client->len;
        int eof = to_client->eof;
```

```
        if (eof) {
            must_sem_post(s1_free);
            must_sem_wait(s2_free);
            to_server->len = 0;
            to_server->eof = 1;
            must_sem_post(s2_ready);
            break;
        }
        char *line = malloc(len + 1);
        if (!line) {
            must_sem_post(s1_free);
            must_sem_wait(s2_free);
            to_server->len = 0;
            to_server->eof = 1;
            must_sem_post(s2_ready);
            break;
        }
        memcpy(line, to_client->data, len);
        line[len] = '\0';
        must_sem_post(s1_free);           /* слот свободен, server может писать */

        size_t check_len = len;
        if (check_len > 0 && line[check_len - 1] == '\n') check_len--;

        int valid = 0;
        if (check_len > 0) {
            char last = line[check_len - 1];
            if (last == '.' || last == ';') valid = 1;
        }

        if (valid) {
            if (write(STDOUT_FILENO, line, len) != (ssize_t)len) {
                write(STDERR_FILENO, "write failed\n", 13);
                _exit(1);
            }
        } else {
            const char *pfx = "INVALID: ";
            size_t plen = strlen(pfx);
            int need_nl = (len == 0 || line[len - 1] != '\n');

            must_sem_wait(s2_free);
            size_t off = 0;
            if (plen > MAX_LINE) plen = MAX_LINE;
            memcpy(to_server->data, pfx, plen); off += plen;
            size_t copy = len;
            if (off + copy > MAX_LINE) copy = MAX_LINE - off;
            memcpy(to_server->data + off, line, copy); off += copy;
            if (need_nl && off < MAX_LINE) to_server->data[off++] = '\n';
            to_server->len = off;
            to_server->eof = 0;
            must_sem_post(s2_ready);
        }
        free(line);
```

```
    }

    munmap(to_client, sizeof(struct slot));
    munmap(to_server, sizeof(struct slot));
    sem_close(s1_ready); sem_close(s1_free);
    sem_close(s2_ready); sem_close(s2_free);
    return 0;
}
```

Протокол работы программы

Тестирование:

./server

Enter file name: a1.txt

werw.

wre

qwees;

3e

a1.txt:

werw.

qwees;

INVALID: wre

INVALID: 3e

Вывод

Были проблемы с организацией передачи данных, было очень интересно переделывать прежнюю лабораторную работу, но выполнять условие другим способом. Также занимательным является разбираться в взаимодействии 2 программ. Надеюсь, это пригодится в моей будущей работе.